GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India

SMART RESUME SCREENER & ATS FEEDBACK PORTAL

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Submitted By: Hemant Tyagi

Roll No.: 22515500039 | Section: NE (L1)

Under the Supervision of: <SUPERVISOR NAME>

Reviewed By: <REVIEWER NAME>

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SEPTEMBER, 2026

Revision History

Version	Date	Change	Reason
1.0	04-09-2026	Initial version	First submission

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) defines the requirements of the Smart Resume Screener & ATS Feedback Portal. The document specifies what the system will do, the technologies and components involved, the expected inputs and outputs, and the criteria used to verify that the implementation satisfies the project objectives.

1.2 Scope

The system is a dual-sided recruitment portal for job seekers and recruiters. A candidate can provide a PDF resume and a target job description. The system extracts resume text, compares it with the job description using TF-IDF and Cosine Similarity, evaluates direct skill overlap using the Jaccard Similarity Index and keyword frequency, and produces a transparent match score from 0–100%.

The portal also provides missing-skill feedback and resume improvement suggestions for candidates. Recruiters can review candidate scores, compare skills, rank candidates, and view graphical analytics. The system does not automatically apply for jobs, send recruitment emails, schedule interviews, or perform OCR on scanned PDFs in Version 1.0.

1.3 Definitions

Term	Definition
ATS	Applicant Tracking System used to support automated resume screening and recruitment workflows.
ATS Score	A 0–100% compatibility rating generated by the system from text similarity, skill overlap, and normalized keyword frequency.
Skill Gap	A list of job-related skills found in the job description but not adequately matched in the resume.
TF-IDF	A text-vectorization technique used to represent the importance of terms in documents.
Cosine Similarity	A mathematical measure used to determine similarity between the resume and job-description TF-IDF vectors.
Jaccard Index	A set-based similarity measure used to evaluate overlap between identified skills/keywords.
Normalized Frequency	A scaled measure of keyword occurrence used as one component of the final score.
Candidate	A job seeker who uses the portal to analyze and improve a resume.
Recruiter / HR	A hiring user who screens, compares, and ranks candidates.

2. Overall Description

2.1 User Classes

User class	Who they are	What they may do
Student / Candidate	Authenticated job seeker.	Upload a PDF resume, provide a job description, view ATS score, inspect skill gaps, receive improvement suggestions, and track score changes across resume versions.
Recruiter / HR	Authenticated hiring user.	Provide job details, screen candidates, view compatibility scores, rank candidates, compare skills, and view analytical charts.

2.2 Assumptions and Dependencies

• The application is designed as a web-based system requiring a modern browser.

• The Java processing component uses Java 17, basic OOP concepts, and Apache PDFBox for PDF text extraction.

• The Python FastAPI service performs text processing, TF-IDF vectorization, Cosine Similarity, Jaccard-based skill comparison, and weighted score calculation.

• PlanetScale (MySQL) is the planned cloud database for candidate profiles, job descriptions, parsed skills, and score histories.

• The frontend uses HTML5, Native CSS3, Vanilla JavaScript (ES6+), and Chart.js.

• Docker is used to containerize the application components for deployment on Render.

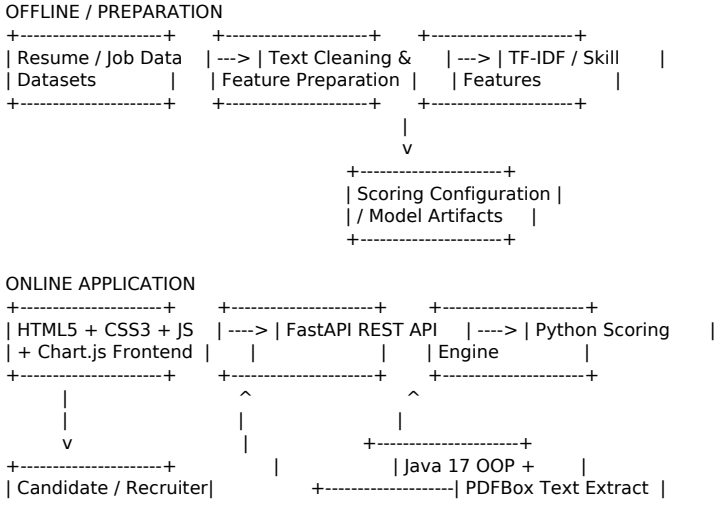
2.3 Constraints

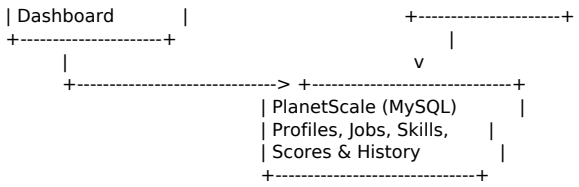
Constraint	Description
C1	Version 1.0 supports PDF resumes; the maximum upload size shall be configured by the deployed application.
C2	Image-only scanned PDFs are not supported because OCR is outside the scope of Version 1.0.
C3	The system is intended to use lightweight TF-IDF and mathematical scoring rather than large transformer-based models.
C4	The initial implementation focuses on English-language resume and job-description text.
C5	Multi-tenant enterprise-level role-based authorization is outside the scope of Version 1.0.
C6	The ATS score is an assistive ranking/matching indicator and does not make an automatic final hiring decision.

3. Data Source and Baseline

Item	Value
Dataset name	Resume and Job Description datasets used for development and evaluation.
Source and URL	Public dataset source(s) selected for the project. The final report shall list the exact dataset name and source URL used.
Licence / terms of use	The project shall follow the licence and terms specified by the exact dataset source used.
Data used	Resume text, job-description text, extracted skills/keywords, and derived matching features.
Prediction / scoring target	Transparent candidate compatibility score on a 0–100% scale.
Task type	Text similarity and mathematical/rule-based scoring using TF-IDF, Cosine Similarity, Jaccard Index, keyword frequency, and weighted averages.
Primary evaluation metric	MAE may be used if reference numerical scores are available; otherwise component-level validation and consistency checks shall be used.
Why that metric	MAE is appropriate for measuring the average absolute difference between a generated score and an available reference score.
Baseline	A basic keyword-overlap or fixed/average-score baseline may be used for comparison before applying the combined scoring formula.

4. System Architecture





Component

Component	Responsibility
Java Processing Engine	Uses Java 17, basic OOP classes, and Apache PDFBox to extract raw text from PDF resumes.
FastAPI Service	Provides REST endpoints and runs the Python-based text processing and scoring workflow.
Scoring Engine	Calculates TF-IDF vectors, Cosine Similarity, Jaccard skill overlap, normalized keyword frequency, and the weighted final score.
Database Store	PlanetScale (MySQL) stores candidate profiles, job descriptions, parsed skill information, and score histories.
Web Dashboard	HTML5, Native CSS3, Vanilla JavaScript, and Chart.js provide score gauges, radar charts, comparison bar graphs, and candidate views.
Docker / Render	Docker packages the application for reproducible deployment, with the production deployment planned on Render.

5. Functional Requirements

5.1 Data Ingestion

FR-1.1 The system shall allow a user to upload a PDF resume.

FR-1.2 The system shall validate the uploaded file type before processing.

FR-1.3 The Java processing component shall use Apache PDFBox to extract readable text from supported PDF resumes.

FR-1.4 The system shall report an understandable error when no readable text can be extracted.

FR-1.5 The system shall accept a job description as text input for resume comparison.

5.2 Text Processing and Matching

FR-2.1 The system shall preprocess resume and job-description text before feature generation.

FR-2.2 The system shall generate TF-IDF vectors for the resume and job description.

FR-2.3 The system shall calculate Cosine Similarity between the generated text vectors.

FR-2.4 The system shall identify relevant skills and keywords from the resume and job description.

FR-2.5 The system shall calculate skill overlap using the Jaccard Similarity Index.

FR-2.6 The system shall calculate a normalized keyword-frequency component.

5.3 ATS Scoring and Feedback

FR-3.1 The system shall calculate a final candidate match score on a 0–100% scale.

FR-3.2 The final score shall combine Cosine Similarity, Jaccard Index, and normalized keyword frequency using configurable weights.

FR-3.3 The system shall display the overall score and its relevant component information.

FR-3.4 The system shall identify and display missing or weakly matched skills.

FR-3.5 The system shall provide basic resume improvement suggestions based on identified skill gaps.

5.4 Candidate Features

FR-4.1 The candidate shall be able to view the result of a resume analysis.

FR-4.2 The candidate shall be able to review missing-skill feedback.

FR-4.3 The system shall maintain score history for supported resume versions or repeated analyses.

FR-4.4 The candidate dashboard shall display the analysis results using readable score and feedback components.

5.5 Recruiter Features

FR-5.1 The recruiter shall be able to provide job details and a job description.

FR-5.2 The recruiter shall be able to screen candidate resumes against a selected job description.

FR-5.3 The system shall calculate compatibility scores for screened candidates.

FR-5.4 The recruiter shall be able to rank candidates using their generated compatibility scores.

FR-5.5 The recruiter dashboard shall provide comparative visualizations for candidate skills and score information.

5.6 Data Visualization

FR-6.1 The system shall display a visual representation of the candidate's ATS score.

FR-6.2 The system shall provide comparison bar graphs for relevant candidate/skill information.

FR-6.3 The system shall provide radar-chart analysis where sufficient skill data is available.

6. Non-Functional Requirements

NFR-1 The system should provide analysis results within a reasonable response time under normal deployment conditions.

NFR-2 The system shall validate uploaded files and sanitize user-provided text to reduce basic security risks.

NFR-3 The system shall maintain consistent results for the same resume, job description, and scoring weights.

NFR-4 The system shall use a layered and modular architecture so that Java processing, FastAPI APIs, scoring logic, database operations, and frontend components can be maintained separately.

NFR-5 The system shall use Docker for consistent application packaging and deployment.

NFR-6 The system shall record relevant analysis history and processing information required for application operation.

NFR-7 The system shall provide a usable web interface for both candidate and recruiter workflows.

7. Build Plan

7.1 Minimum Viable Product (MVP)

The MVP includes PDF upload and validation, Java/PDFBox text extraction, FastAPI REST APIs, TF-IDF and Cosine Similarity, Jaccard skill overlap, normalized keyword-frequency scoring, a weighted 0–100% ATS score, missing-skill feedback, a basic candidate/recruiter dashboard, PlanetScale MySQL persistence, and Docker-based deployment.

7.2 Timeline

Stage	What will be working
Weeks 1–2: Setup & Schema Design	Configure repositories, project structure, and PlanetScale MySQL schema.
Weeks 3–4: Java Processing Engine	Develop Java 17 OOP parser classes and integrate Apache PDFBox for PDF text extraction.
Weeks 5–7: ML & Scoring Engine	Implement TF-IDF, Cosine Similarity, Jaccard skill overlap, normalized frequency, and weighted scoring formulas.
Weeks 8–9: API & Software Engineering	Develop FastAPI REST endpoints, layered architecture, integration between components, and Docker configuration.
Weeks 10–11: Web UI & Analytics	Build HTML/CSS/JavaScript frontend and integrate Chart.js score gauges, radar charts, and comparison graphs.
Weeks 12–13: Deployment & Testing	Deploy the containerized application on Render, conduct end-to-end testing, fix defects, and finalize documentation.

8. Data Requirements

Entity: CandidateProfile

• id: Unique candidate identifier

• candidate_name: Candidate name

• contact/profile information required by the application

Entity: ResumeRecord

• id: Unique resume identifier

• candidate_id: Reference to the candidate

• file_name: Uploaded PDF file name

• raw_text: Text extracted from the PDF

• created_at: Resume upload/record timestamp

Entity: JobDescription

• id: Unique job identifier

• job_title: Job title

• description_text: Job description

• required_skills: Parsed or configured job skills

Entity: AnalysisResult

• id: Unique analysis identifier

• resume_id: Reference to the analyzed resume

• job_id: Reference to the selected job description

• cosine_score: TF-IDF/Cosine Similarity component

• jaccard_score: Skill-overlap component

• normalized_frequency: Keyword-frequency component

• ats_score: Final 0–100% weighted score

• matched_skills: Skills identified as matched

• missing_skills: Skills identified as missing/weak

• created_at: Analysis timestamp

9. Design Decisions and Alternatives Considered

#	Decision	Alternative rejected	Reason
D1	TF-IDF + Cosine Similarity	BERT/large transformer model	Lightweight, explainable, easier to implement, and suitable for the project's resource and complexity constraints.
D2	FastAPI	Django or Spring Boot as primary API framework	FastAPI provides a lightweight Python API layer and direct integration with the selected scoring libraries.
D3	Java 17 + PDFBox for PDF extraction	Python-only extraction	The project specifically demonstrates Java OOP and uses PDFBox as the Java PDF-processing component.
D4	Jaccard + normalized frequency	Complex semantic-ranking model	Simple mathematical components are transparent and align with the project's Applied Math & Statistics objectives.
D5	PlanetScale (MySQL)	Local file/CSV storage	Provides cloud-hosted relational persistence for profiles, jobs, skills, and score histories.

#	Decision	Alternative rejected	Reason
D6	HTML5 + CSS3 + Vanilla JS + Chart.js	Heavy frontend framework	Keeps the frontend straightforward while supporting interactive analytics and visualizations.
D7	Docker + Render	Manual server deployment	Provides reproducible packaging and a straightforward cloud deployment workflow.

10. Acceptance Criteria

AC-1 A valid PDF resume can be uploaded and processed by the Java/PDFBox extraction component.

AC-2 An invalid or unsupported upload is rejected with a clear error message.

AC-3 A valid resume and job description produce a deterministic score on a 0–100% scale.

AC-4 The analysis result contains a Cosine Similarity component, Jaccard skill-overlap component, and normalized keyword-frequency component.

AC-5 Missing or weakly matched skills are displayed to the candidate.

AC-6 A recruiter can screen and rank candidates using the generated compatibility scores.

AC-7 The candidate/recruiter interface displays the required charts using Chart.js.

AC-8 Candidate, job, skill, and analysis information can be stored and retrieved from PlanetScale MySQL.

AC-9 The application can be packaged with Docker and deployed on Render.

11. Out of Scope for This Release

• Optical Character Recognition (OCR) for scanned or image-only PDFs.

• Automated candidate email notifications.

• Automated interview scheduling.

• Automatic job application submission.

• Multi-tenant enterprise-level role-based authorization (RBAC).

• Large transformer/LLM-based resume ranking.

• Automatic final hiring or rejection decisions without human review.

• Broad multilingual resume processing.

Appendices

Appendix A: State Model – N/A for Version 1.0.

Appendix B: Traceability – The MVP requirements are mapped to the project phases for Java PDF processing, ML/scoring, FastAPI and software-engineering implementation, web UI/data visualization, database persistence, testing, and deployment.

Subject Coverage: Machine Learning – Scikit-Learn, TF-IDF Vectorizer, Cosine Similarity; Applied Math & Statistics – Jaccard Similarity Index, Min-Max Normalization, Term Frequency, Weighted Averages; Full-Stack Development – FastAPI, HTML5, CSS3, JavaScript, PlanetScale MySQL; Software Engineering – REST APIs, Layered Architecture, Docker, Render, Git; Data Visualization – Chart.js; Java & OOP – Java 17, Classes, Inheritance, Polymorphism, Apache PDFBox.